Document number: P0091R2
Revision of P0091R1
Date: 2016-05-29
Reply-To:
Mike Spertus, Symantec (mike_spertus@symantec.com)
Faisal Vali (faisalv@yahoo.com)
Richard Smith (richard@metafoo.co.uk)
Audience: Core Working Group

Template argument deduction for class templates (Rev. 5)

Changes from P0091R1

- Added wording
- Discuss interaction with universal references

Summary

This paper proposes extending template argument deduction for functions to constructors of template classes and incorporates feedback from the EWG review of P0091R0 and from implementation experience.

Currently, if we want to construct template classes, we need to specify the template arguments. For example, N4498 on Variadic Lock Guards gives an example of acquiring a `lock_guard` on two mutexes inside an `operator=` to properly lock both the source and destination of the assignment. Expanding typedefs, the locks are acquired by the following statement (See the paper for details and rationale).

```
std::lock_guard<std::shared_timed_mutex, std::shared_lock<std::shared_timed_mutex>> lck(mut_, r1);
```

Having to specify the template arguments adds nothing but complexity! If constructors could deduce their template arguments "like we expect from other functions and methods," then the following vastly simpler and more intuitive code could have been used instead.

```
auto lock = std::lock_guard(mut_, r1);
```

The sections below first spell out the problem in more detail and then makes precise what "like we expect from other functions and methods" means in this context.

The problem

To simplify the examples below, suppose the following definitions are in place.

```
vector<int> vi1 = { 0, 1, 1, 2, 3, 5, 8 };
vector<int> vi2;
std::mutex m;
unique_lock<std::mutex> ul(m, std::defer_lock);
template<class Func>
class Foo() {
public:
    Foo(Func f) : func(f) {}
    void operator()(int i) {
      os << "Calling with " << i << endl;
      f(i);
    }
private:
    Func func;
    mutex mtx;
};
```

In the current standard, the following objects would be constructed as shown

```
pair<int, double> p(2, 4.5);
auto t = make_tuple(4, 3, 2.5);
copy_n(vi1, 3, back_inserter(vi2));
// Virtually impossible to pass a lambda to a template class' constructor without declaring the lambda
for_each(vi2.begin(), vi2.end(), Foo<???>([&](int i) { ...}));
lock_guard<std::mutex> lck(foo.mtx);
lock_guard<std::mutex, std::unique_lock<std::mutex>> lck2(foo.mtx, ul); // Notation from N4470
auto hasher = [](X const & x) -> size_t { /* ... */ };
unordered_map<X, int, decltype(hasher)> ximap(10, hasher);
```

There are several problems with the above code:

- Creating "make functions" like `make_tuple` is confusing, artificial, extra boilerplate, and inconsistent with how non-template classes are constructed.
- Since the standard library doesn't follow any consistent convention for make functions, users have to scurry through documentation to find they need to use `make_tuple` for `tuple`s and `back_inserter` for `back_insert_iterator`s. Of course their own template classes might not be as consistent or thoroughly-documented as the standard.
- Specifying template arguments as in `pair<int, double>(2, 4.5)` should be unnecessary since they can be inferred from the type of the arguments, as is usual with template functions (this is the reason the standard provides make functions for many template classes in the first place!).
- A make function may do more than just deduce constructor template arguments. Unless a detailed study of the documentation is made (which usually only happens when debugging weird unexpected behavior...), subtle changes in semantics may occur. Committee members and other C++ experts are invited to see if they can tell which of the above make functions simply do the "obvious" deduction of template arguments.
- If we don't have a make function, we may not be able to create class objects without prior declarations of lambdas as indicated by the `???` in the code above.
- Using make functions with classes that aren't movable like `std::lock_guard` requires appealing to highly abstruse language features like copy elision for prvalues (P0135R0), contributing to the perception that C++' conceptual model is beyond ordinary mortals.
- The useful technique of replacing a large function with a class by organizing its code into methods to reduce cyclomatic complexity can't be used for template functions

If we allowed the compiler to deduce the template arguments for constructors of template classes, we could replace the above with:

```
pair p(2, 4.5);
tuple t(4, 3, 2.5);
copy_n(vi1, 3, back_insert_iterator(vi2));
for_each(vi.begin(), vi.end(), Foo([&](int i) { ...})); // Now easy instead of virtually impossible
auto lck = lock_guard(foo.mtx);
```

```
lock_guard lck2(foo.mtx, ul);
unordered_map<X, int> ximap(10, [](X const & x) -> size_t { /* ... */ }); // NOTE: template argument deduction deduces the non-explicitly specified template arguments
```

We believe this is more consistent and simpler for both users and writers of template classes, especially for user-defined classes that might not have carefully designed and documented make functions like `pair`, `tuple`, and `back_insert_iterator`.

The Solution

We propose to allow a template name referring to a class template as a simple-type-specifier or with partially supplied explicit template arguments in two contexts:

- Simple declarations of variables (or variable templates) that are also definitions whose declarator is a *noptr-declarator* (i.e. not when declaring functions, template parameters, function parameters, non-static data members, pointers, references etc.), and

```
template<class ... Ts> struct X { X(Ts...) };
X x1{1}; // OK X<int>
X x11; // OK X<>
template<class T> X xv{(T*)0}; // OK decltype(xv<int>) == X<int*>
extern X x2; // NOT OK, needs to be a definition
X arr[10];   // OK X<>
X x1{1}, x2{2}; // OK, deduced to the same type X<int>
X<int> x3{1, 'a', "bc"}; // OK X<int,char,const char*>
X *pointer = 0; // NOT OK
X &&reference = X<int>{1};
X function(); // NOT OK
```

- Explicit type conversion (functional notation) ([expr.type.conv])

We propose two techniques to support template argument deduction for class templates:

- Using implicitly synthesized deduction guides (from existing constructors)
- Using explicitly specified deduction guides

These techniques can work well together as will be explained below. They can also be adopted separately. Both forms received consensus straw polls from EWG (although note that the authors did not vote unanimously in support of implicit guides).

Implicitly synthesized Deduction Guides (from existing constructors)

In the case of a function-notation type conversion (e.g., "`tuple(1, 2.0, false)`") or a direct parenthesized or braced initialization, the initialization is resolved as follows. First, constructors and constructor templates declared in the named template are enumerated. Let Ci be such a constructor or constructor template; together they form an overload set. A parallel overload set (i.e. the implicitly synthesized deduction guides) *F* of function templates is then created as follows:

For each Ci a function template is constructed with template parameters that include both those of the named class template and if Ci is a constructor template, those of that template (default arguments are included too) -- the function parameters are the constructor parameters, and the return type is the template-name followed by the template-parameters of the class template enclosed in <>

Deduction and overload resolution is then performed for an invented call to *F* with the parenthesized or braced expressions used as arguments. If that call doesn't yield a "best viable function", the program is ill-formed. Otherwise, the return type of the selected *F* template specialization becomes the deduced class template specialization.

Let's look at an example:

```
template<typename T> struct UniquePtr {
    UniquePtr(T* t);
    ...
};

UniquePtr dp{new auto(2.0)};
```

In the above example, `UniquePtr` is missing template arguments in the declaration of 'dp' so they have to be deduced. To deduce the initialized type, the compiler then creates an overload set as follows:

```
    template<typename T>
        UniquePtr<T> F(UniquePtr<T> const&);
    template<typename T>
        UniquePtr<T> F(UniquePtr<T> &&);
    template<typename T>
        UniquePtr<T> F(T *p);
```

Then the compiler performs overload resolution for a call "`F(2.0)`" which in this case finds a unique best candidate in the last synthesized function and after final substitution, deduces the class template specialization as `UniquePtr<double>`

Let's look at a more involved example:

```
template<typename T> struct S {
  template<typename U> struct N {
    N(T);
    N(T, U);
    template<typename V> N(V, U);
  };
};

S<int>::N x{2.0, 1};
```

In this example, "`S<int>::N`" in the declaration of `x` is missing template arguments, so the approach above kicks in. Template arguments can only be left out this way from the "type" of the declaration, but not from any name qualifiers used in naming the template; i.e., we couldn't replace "`S<int>::N`" by just "`S::N`" using some sort of additional level of deduction. To deduce the initialized type, the compiler now creates an overload set as follows:

```
    template<typename U>
        S<int>::N<U> F(S<int>::N<U> const&);
    template<typename U>
        S<int>::N<U> F(S<int>::N<U> &&);
    template<typename U>
        S<int>::N<U> F(int);
    template<typename U>
        S<int>::N<U> F(int, U);
    template<typename U, typename V>
```

```
        S<int>::N<U> F(V, U);
```

(The first two candidates correspond to the implicitly-declared copy and move contructors. Note that template parameter `T` is already known to be `int` and is not a template parameter in the synthesized overload set.) Then the compiler performs overload resolution for a call "`F(2.0, 1)`" which in this case finds a unique best candidate in the last synthesized function with `U = int` and `V = double` and deduced type of `S<int>::N<int>`. The initialization is therefore treated as "`S<int>::N<int> x{2.0, 1};`"

Note that after the deduction process described above the initialization may still end up being ill-formed. For example, a selected constructor might be inaccessible or deleted, or the selected template instance might have been specialized or partially specialized in such a way that the candidate constructors will not match the initializer.

The case of a *simple-declaration* with copy-initialization syntax is treated similarly to the approach described above, except that explicit constructors and constructor templates are ignored, and the initializer expression is used as the single call argument during the deduction process.

Explicitly specified Deduction Guides

While the above procedure generates many useful deducible constructors, some constructors that we would like to be deducible are not. For example, one could imaging a function `make_vector` defined as follows:

```
template<typename Iter>
vector<Iter::value_type> make_vec(Iter b, Iter e) {
  return vector<Iter::value_type>(b, e);
}
```

Although there is no constructor in `vector` from which we can deduce the type of the vector from two iterators, one would like to be able to deduce the type of the vector from the value type of the two iterators. For example, some implementations of the STL define their `value_type` `typedef` as follows

```
template<typename T, typename Alloc=std::allocator<T>>
struct vector {
  struct iterator {
    typedef T value_type;
    /* ... */
  };
  typedef iterator::value_type value_type;
  /* ... */
};
```

The detour through `vector<T>::iterator` keeps us from deducing that `T` is `char` in a constructor call like `vector(5, 'c')`. We would certainly like constructors like that to work.

We suggest a notation to allow explicit specification of a deduction guide in the same semantic scope as the class template using the following syntax:

```
template<typename T, typename Alloc = std::allocator<T>> struct vector {
  /* ... */
};
template<typename Iter> vector(Iter b, Iter e) -> vector<typename iterator_traits<Iter>::value_type>
```

In effect, this allows users to leverage all the deduction rules that are specifiable by any function with a standard first-class name and no boilerplate code in the body. It also allows us to suppress a standard deduction from the above process via "`= delete;`"

Note that a deduction guide is not a function and shall not have a body. It participates in deduction of class template arguments in a similar way to the synthesized deduction guides.

Additionally, it is worthwhile to note the following:

- Deduction guides must name a class template and must be introduced within the same semantic scope of the class template, but they do not becomes members of that scope (i.e if that scope is a class)

Alternate notation

As an alternative to the above notation for explicit deduction guides, one could consider a “declaration notation”. E.g.,

```
template<typename Iter> vector<typename iterator_traits<Iter>::value_type> vector(Iter b, Iter e);
template<typename Iter> auto vector(Iter b, Iter e) -> vector<typename iterator_traits<Iter>::value_type>;
```

The point is that this uses a familiar notation to declare what the canonical “make function” would look like as a function declaration. As the above lines are not legal C++14 (cf. §14p5), this would not reinterpret existing function declarations, and the compiler or linker would not need to check for a function definition. This could potentially be easier for programmers to learn as they do not need to learn a new grammatical construct, and there is indeed an “function” instantiated to match the declaration. The downside of the “declaration notation” is of course the quotes around “function”, as this is not actually a function declaration.

The prototype uses “declaration notation”, but we believe there are no technical parsing obstacles to either (always a relief when defining new C++ features!) and that it is a matter of the preference of the committee.

A note on injected class names

The focus on this paper is on simplifying the interface of a class for its clients. Within a class, one may need to explicitly specify the arguments as before due to the injected class name:

```
template<typename T> struct X {
  template<typename Iter>
  X(Iter b, Iter e) { /* ... */ }

  template<typename Iter>
  auto foo(Iter b, Iter e) {
     return X(b, e); // X<U> to avoid breaking change
  }

  template<typename Iter>
  auto bar(Iter b, Iter e) {
     return X<Iter::value_type>(b, e); // Must specify what we want
  }
};
```

Code compatibility

While we cannot say whether it is a problem in practice, we should point out a scenario where auto-deduction can break compatibility.

Suppose I produce a library and I'm under license to preserve source compatibility across all 1.x upgrades, and I have this class template in version 1.0:

```
template struct X {
  X(T);
};
```

... and in version 1.1 I rewrite it as this:

```
template struct X {
  struct iterator { typedef T type; };
  X(typename iterator::type);
};
```

If one of my users upgrades to C++17, with this change in the language, I am no longer complying with the terms of my licensing. Likewise, if this language change happens between me releasing 1.0 and 1.1, I can no longer release version 1.1 because it might break some of my existing customers.

The point is: current code does not express any intent about whether class template parameters are deducible, based on whether they use the version 1.0 code or the version 1.1 code. But this change makes that implicit property into part of the de facto interface of the code.

Pros and cons of implicit deduction guides

In light of the above, we think it is worth calling out the benefits and costs of providing implicit deduction guides versus requiring explicit deduction guides everywhere

Basically, having to manually specify boilerplate for what is obviously expected has an insidious cost as any (honest) Java programmer can tell you. There are natural implementations of all of the examples in The Problem section above where only implicit deduction guides are necessary. (Alternate implementations of those classes may require explicit guides but do not create unnatural deductions). As many classes have dozens of constructors, not only is creating myriad explicit deduction guides tedious and error-prone but will (predictably) drift out of sync with the actual constructors as the class evolves. While not suitable for all purposes, this is a much-requested feature to simplify routine programming (cf. range-based `for`) and current practice or explicit deduction guides remain available (see next paragraph for exceptions) if the implicit deduction is not sufficient, mitigating downside.

So what is the cost of implicit deduction guides? The Code compatibility section above shows that equivalent code in C++14 may no longer be equivalent in C++17 (Note that this example does not change the behavior of C++14 code when compiled with a C++17 compiler). This particular incompatibility can be rectified by adding explicit deduction guides as needed.

Another cost of implicit deduction guides is that they may trigger instantiations that cause hard errors. For example, consider the following class

```
template<class T> struct X {
     using ty = T::type;
     static auto foo() { return typename T::type{} };
     X(ty); #1
     X(decltype(foo())); #2
     X(T);
  };

  template<class T>
  struct X<T*> {
    X(...);
  };
```

For such a class, the prototype implementation allows

```
X x{(int *)0};
```

but normal instantiation rules suggest a hard error. We plan to discuss implications with the committee. Note that the current

```
X<int *> x{0}
```

remains legal.

Universal reference interactions

By deducing the template arguments, sometimes and rvalue reference can be reinterpreted as a universal reference as the following example due to Sebastian Gesemann shows.

```
template<class T>
    struct Wrapper
    {
        T value;
        Wrapper(T const& x): value(x) {}
        Wrapper(T && y): value(std::move(x)) {}
    };

    int main()
    {
        std::string foo = "Hello";
        auto w = Wrapper(foo);           // Error
    }
```

In the implicit deduction guide for the second constructor, `T &&` is now interpreted as a universal reference rather than an rvalue reference, resulting in a failure to find a valid match. This does not seem to result in a dangerous deduction but rather a failure to compile, but Core should consider this case. The resolution is to write an explicit deduction guide will need to be written to explain the intent.

```
template<typename T> Wrapper(T &&y) -> Wrapper<remover_reference_t<T>>;
```

Of course, traditional constructor invocation without deduction will continue to work as well.

Wording

Insert a paragraph after §3.4p3 [basic.lookup]:

> If the lookup finds the name of a template class and is not the injected-class-name, then it also finds all sythesized functions associated with deduction guides for that template class (14.9).

Change §7p1[dcl.dcl] as follows

declaration:
block-declaration
nodeclspec-function-declaration
function-definition
template-declaration
explicit-instantiation
linkage-specification
namespace-definition
empty-declaration
attribute-declaration
deduction-guide
block-declaration

Also, change the note at the bottom of §7p1 [dcl.dcl] as follows

[*Note: asm-definitions* are described in 7.4, and *linkage-specifications* are described in 7.5. *Function-definitions* are described in 8.4 and *template-declarations* and *deduction-guides* are described in Clause 14.

At the end of clause 14 [Template], add a new section

14.9 Deduction guides

Deduction guides synthesize functions associated with constructors of class templates to participate in template argument deduction(14.8.2) similarly to function templates. Deduction guides may be either *implicit* or *explicit*.

The compiler generates an implicit deduction guide for every constructor in a primary class template. The synthesized function associated with a constructor in the primary class template has the template parameters of the class primary template followed by any template parameters from the constructor. The function parameters for the synthesized function are the same as for the constructor and the return type is that of the class template, and the effect of the function is to return the object constructed by calling that constructor.

[*Example:*

```
template<class T> class X { X(T t) { /* ... */}

void f() {
  auto x = X(7); // T is deduced to int
}
```

[*— end example*]

Explicit deduction guides declare constructors that participate in template argument deduction. The

deduction-guide:
`template` `<` *template-parameter-list* `>` *class-name* `(` *parameter-declaration-clause* `)` `->` *simple-template-id*

In a *deduction-guide*, the *class-name* and the *simple-type-id* must refer to the same class template, denoted as the *class template of the deduction guide*. The synthesized function associated with the deduction guide has the same template and function parameters as the deduction guide, and the effect of the deduction guide is to construct an object of the return type by forwarding its function arguments to the constructor. The deduction guide must be introduced in the same semantic scope as the class template.